

TEORÍA DE ALGORITMOS 1

Algoritmos heurísticos y búsqueda local

Por: ING. VÍCTOR DANIEL PODBEREZSKI
vpodberezski@fi.uba.ar

1. Introducción

Se conoce como algoritmos heurísticos a aquellos que utilizan una o varias heurísticas para llegar a una solución de un problema. Podemos definir **heurística** como un proceso de toma de decisiones que se basa en experiencias previas en problemas similares. Muchos métodos se basan en simular o emular procesos de la naturaleza. Este proceso permite obtener un resultado que no garantiza ser la solución óptima de la instancia analizada (o inclusive una solución factible) aunque intenta acercarse lo más posible a ella. En ocasiones ni siquiera podemos saber que tan cerca de la solución óptima se encuentra una solución factible encontrada. Se espera que esta búsqueda se desarrolle en un tiempo razonable. Por lo tanto se suele aplicar a problemas donde no se conoce un algoritmo bueno o que requiera un tiempo elevado de ejecución.

Dentro de los algoritmos heurísticos se encuentran los de **búsqueda local**. En estos la resolución del problema se realiza explorando el conjunto de soluciones del mismo. Podemos pensar esta exploración como un recorrido en grafo de espacio de estados del problema. Iniciando en un posible estado inicial, que puede corresponder a una solución factible o una solución parcial, se evalúan un subconjunto de estados para buscar un estado que mejore la situación actual. Dado un criterio de elección se establece el siguiente estado y en un proceso iterativo se vuelve a realizar la evaluación de aquellos estados accesibles desde este. Dado que no se garantiza encontrar la solución óptima y que se desea que el proceso no se extienda en el tiempo, generalmente se establece un criterio de finalización de la ejecución. Puede ser una cantidad determinada de iteraciones o llegar a un estado en el que sea imposible mejorar la solución previamente encontrada.

El paso inicial para la creación de un algoritmo de búsqueda local consiste en determinar cómo representar cada estado posible del problema. En problemas combinatorios un estado del problema corresponde a la elección o agrupación de un subconjunto de elementos donde puede importar o no el orden o agrupación del mismo. Al trabajar la resolución de problemas por fuerza bruta (generar y probar, Backtracking y branch and bound) estudiamos diversas alternativas para la representación de estados del problema. Estos mismos se pueden utilizar, aunque no se limitan a estos, también se puede aplicar los algoritmos de búsqueda local a otros tipos de problemas donde la solución se puede expresar como un conjunto de variables cuantitativas.

Se desprende la necesidad de poder comparar entre soluciones. De forma tal que podamos determinar entre un conjunto de ellas la más conveniente. Llamaremos **función costo** al valor numérico que caracteriza un estado del problema. En los problemas de optimización esta función suele estar explicitada por el mismo enunciado. Sin embargo es posible utilizarla en conjunto con las restricciones implícitas. Para problemas de exploración al no existir un valor a maximizar (o minimizar) se puede construir la función costo contabilizando la cantidad de restricciones violadas por la solución.

El nombre de la metodología “búsqueda local” se lo brinda el mecanismo de restringir entre un subconjunto de estados próximos el posible seleccionado. Llamaremos **función de vecindario** (neighborhood function) a aquella que define los posibles estados sucesores para un estado determinado. Existen numerosas alternativas para definir esta función. Generalmente se busca que el estado próximo sea una variación del anterior aceptando una cantidad acotada de modificaciones. Se suele hablar de distancia entre estados y muchas veces se define a la función vecindario como el subconjunto de todos aquellos que no se encuentran a una mayor distancia que el estado actual. Por ejemplo, si la solución es un conjunto de elementos seleccionados, la distancia entre dos estados se podría medir como todas aquellas diferencias (elementos faltantes o agregados) entre ellos. Si la solución es una permutación entre elementos la distancia se podría determinar como la cantidad de intercambios de elementos necesarios para llegar de uno a otro.

La manera de realizar las transiciones entre estados es otra característica clave a la hora de intentar buscar la solución a la instancia mediante un algoritmo de búsqueda local. Una opción, que denominaremos **best improvement**, consiste en recorrer de forma

exhaustiva todos los estados definidos por la función de vecindario y seleccionar aquella que logra un mayor aumento (o disminución) en la función costo. Alternativamente se puede utilizar **first improvement**, donde seleccionamos aleatoriamente un vecino y solo realizar la transición si mejora la función costo con respecto al estado actual. Se puede mitigar la condición de cambio de estado, permitiendo en ocasiones realizarlo si no hay aumento en la función costo o incluso si empeora.

Finalmente se debe establecer un estado inicial para comenzar el proceso. El mismo puede ser seleccionado de forma aleatoria o seleccionado mediante algún proceso algorítmico anterior. En caso de problemas de optimización generalmente el estado inicial corresponde a una solución factible o una solución parcial que cumple con las restricciones del problema. Este estado inicial es clave dado que generalmente condiciona a la porción del espacio de soluciones explorado. Se debe tener en cuenta que los métodos de búsqueda local en raras ocasiones exploran todo el espacio de soluciones. El tamaño de este en muchos casos lo hace prohibitivo si se desea una complejidad temporal polinomial. Por otro lado, en ocasiones por la naturaleza de las instancias una ejecución puede llegar a un estado donde sea imposible encontrar un estado vecino superador. Ese valor se denomina como **máximo (o mínimo) local**. Solo uno de ellos (o un subconjunto) corresponde al máximo global.

Una opción que ciertas implementaciones persiguen es construir un algoritmo veloz y realizar varias ejecuciones modificando el estado inicial. De esa forma intentar hallar el máximo global que aventajara a los máximos locales encontrados. Se conoce a esta propuesta como **aproximación de múltiple inicio** (multistart). Otra elección corresponde a ejecutar varias veces el algoritmo pero modificando la función de vecindario. A esa propuesta se la conoce como **aproximación de múltiple nivel** (multilevel).

La medición de la complejidad espacial y temporal de estos algoritmos generalmente es sencilla, no obstante no ocurre lo mismo con la determinación de la optimalidad del mismo. En ciertos casos es posible calcular un margen de error, pero en el caso general esto no es posible. La mayoría de los estudios al presentar los algoritmos se basan en resultados empíricos.

En las próximas secciones analizaremos diferentes tipos de algoritmos de búsqueda local. En cada uno de ellos se selecciona un problema particular para ejemplificar su funcionamiento.

2. Hill Climbing y el problema de satisfacibilidad booleana

El primero de los algoritmos de búsqueda local que analizaremos es el conocido como de escalada simple o de **ascenso de colinas** (Hill Climbing). Corresponde posiblemente al más sencillo entre los de su clase. Este algoritmo recorre de forma exhaustiva todos los estados accesibles desde el estado actual. Selecciona a aquel factible que mejora en mayor medida la función costo. Es decir que utiliza una estrategia de best improvement. También se la suele denominar como **ascenso más empinado** (steepest ascent) por analogía a ascender a la colina lo más veloz posible. El algoritmo requiere un estado inicial y se repite iterativamente hasta llegar a un estado máximo (mínimo) local o llegar a un número predefinido de iteraciones realizadas. Esta última restricción nos asegura la ejecución acotada en el tiempo y nos permite poder determinar una complejidad temporal preestablecida.

El siguiente es el pseudocódigo del algoritmo genérico:

Algoritmo genérico Hill Climbing
Sea s el estado actual del problema Sea Max la máxima cantidad de iteraciones Sea $it=0$ la cantidad de iteraciones Establecer en s el estado inicial Calcular c el valor de la función costo de s Repetir Buscar s_v el estado vecino de s con mayor función costo Calcular c_v el valor de la función costo de s_v Si $c_v < c$ Definir s como s_v . Definir c como c_v . incrementar it en 1 Mientras que $it < Max$ y haya mejora en función costo

Utilizaremos como ejemplo el problema de la satisfacción booleana. Recordemos su enunciado:

Problema SAT

Dada una expresión booleana ϕ compuesta de n variables y k cláusulas, queremos saber si la misma se puede satisfacer.
--

Para resolver el problema con este método primero definiremos como expresar una posible solución del problema. Dado la existencia de n variables booleanas, podemos representar la solución como un vector de n posiciones. Cada posición puede tomar dos valores "1" o "0" (Verdadero o Falso). El elemento de la posición i en el vector corresponderá al valor asignado a la variable i . Existen 2^n posibles estados.

Podemos ver al problema como un problema de maximización de cláusulas a satisfacer. En ese caso podemos expresar la función costo como la cantidad de cláusulas que se satisfacen dada una asignación de variables determinada.

El siguiente ejemplo corresponde una fórmula booleana expresada en CNF: $\phi = (\neg v_1) \wedge (v_2 \vee v_3) \wedge (v_1 \vee \neg v_3) \wedge (v_1 \vee v_2 \vee v_3)$. Se compone de 3 variables y 4 cláusulas. Podemos expresar una posible solución como un vector de 3 posiciones v_1, v_2, v_3 . Por ejemplo 0,0,0 corresponde a la solución donde las tres variables toman el valor "False". Si evaluamos la función costo - la cantidad de cláusulas que se satisfacen con la asignación - con esta asignación de variables vemos que su valor es 2. La primera y tercera cláusulas se activan. En este caso, una instancia realmente pequeña, se pueden construir 8 alternativas de solución.

La función vecindario de un estado la vamos a definir como todos aquellas asignaciones de variables que se diferencien en solo una de ellas. Podemos ver que cada estado tendrá " n " estados vecinos. Podemos calcular los vecinos de un estado mediante el siguiente pseudocódigo:

Obtener estados vecinos

Sea s el estado actual del problema Sea v los estados vecinos de s $v = \emptyset$
--

```
Desde i=1 a n
    Sea  $s_v = s$ 
    Establecer  $s_v[i] = 1 - s_v[i]$ 
    Agregar  $s_v$  a  $v$ 
Retornar  $v$ 
```

Siguiendo con el ejemplo anterior, el estado 0,0,0 tiene como vecinos a los estados 1,0,0 ; 0,1,0 y 0,0,1. Por otro lado el estado 1,0,1 tiene como vecinos a los estados 0,0,1 ; 1,1,1 y 1,0,0.

Se debe determinar un valor máximo de iteraciones a realizar en la búsqueda del estado máximo. Este valor puede ser un valor prefijado o un valor proporcional a la cantidad de variables. Dependiendo de la elección de uno u otro impactará en la complejidad temporal del algoritmo. Para nuestra formulación tomaremos un valor constante prefijado.

El estado inicial con el que comenzar la ejecución lo seleccionaremos de forma aleatoria. Siendo que contamos con 2^n posibles estados, podemos buscar un valor aleatorio entre 0 y 2^n . Este valor expresado en binario se utiliza para generar el vector del estado actual.

De forma iterativa, comenzando por el estado inicial, se consultará a sus vecinos y se seleccionará a aquel que incremente en más la cantidad de cláusulas satisfechas. Se debe establecer algún mecanismo de desempate en caso 2 o más estados consigan el mismo resultado. Nuevamente existen diversas alternativas. Podría ser: el primero encontrado, el último encontrado, selección aleatoria, si la variable cambiada está en más cláusulas, en menos cláusulas, entre otras. En este caso seleccionaremos una entre todas ellas de forma aleatoria. Para lograrlo ordenaremos los vecinos de forma aleatoria y nos quedaremos con el primer de los estados encontrados con el mayor valor de función costo.

Supongamos, para resolver $\phi = (\neg v_1) \wedge (v_2 \vee v_3) \wedge (v_1 \vee \neg v_3) \wedge (v_1 \vee v_2 \vee v_3)$, que el estado 0,0,0 se tomó entre todos los estados posibles de forma aleatoria como inicial. Su función costo toma el valor de 2. Para cada uno de sus vecinos calculamos el valor costo. 1,0,0 tiene un costo de 2 (se activa la cláusula 3 y 4). 0,1,0 tiene un costo de 4. Por último 0,0,1 tiene un costo 2. La segunda opción es, entre todos los vecinos, la de mayor costo. 0,1,0 pasa a ser el estado actual. Nuevamente se vuelve a calcular sus vecinos y entre ellos el que maximiza la función costo. No será posible superarlo dado que activar 4 cláusulas es lo máximo que se puede lograr con esa misma cantidad. El algoritmo finaliza habiendo encontrado un óptimo local, que en este caso equivale al óptimo global (y a lograr satisfacer la expresión completa).

El conjunto de los criterios seleccionados fue propuesto en 1992 y nombrado como GSAT¹. En pseudocódigo lo podemos expresar como:

GSAT
Sea v el vector con la asignación de variables Sea max como la cantidad máxima de iteraciones a realizar Seleccionar de forma aleatoria una asignación de v Calcular c el valor de la función de costo de v Definir $it=0$ Repetir Obtener vecinos los estados vecinos del estado v Ordenar de forma aleatoria vecinos Sea mejora = 'no' Por cada estado e en vecinos Calcular c_e el valor de la función de costo de e . Si $c_e < c$ $c = c_e$ $v = e$ mejora = 'si' Incrementar it en 1 Hasta que mejora == 'no' o $it > max$ Retornar c, v

Para calcular la complejidad temporal del algoritmo debemos considerar la complejidad de obtener los vecinos $O(\text{"Obtener Vecinos"})$, determinar el valor de la función costo $O(\text{"Calcular costo"})$. La cantidad de vecinos por cada estado es " n ". Por lo tanto La iteración de estados es $O(n * \text{"Calcular costo"})$. Ordenar de forma aleatoria se puede hacer en $O(n)$. La iteración exterior es $O(1)$ al una cantidad constante de veces. Por lo que la complejidad final del proceso se puede establecer como $O(\text{"Obtener Vecinos"} + n * \text{"Calcular costo"})$.

En cuanto a la complejidad espacial, tal como está planteado, se debe reservar espacio para los vecinos. Cada uno es un vector de " n " posiciones y en total existen " n ". Por lo que el espacio adicional requerido es $O(n^2)$.

¹ Selman, B.; Levesque, H.; and Mitchell, D. 1992. A New Method for Solving Hard Satisfiability Problems. In Proc. 10th National Conference on AI. 440-446.

Este algoritmo además de ser de tipo hill climbing es randomizado. Los algoritmos de hill climbing más básicos no utilizan esta técnica y son totalmente determinísticos. En GSAT se utiliza la elección random para desempatar entre vecinos y para seleccionar un estado inicial. Al ser un algoritmo veloz y para aumentar la probabilidad de encontrar la asignación de variables que satisfaga la expresión booleana evitando los máximos locales se puede ejecutar el mismo algoritmo más de una vez. Esta idea se conoce como GenSAT². Este algoritmo contiene como parámetro adicional el número máximo de ejecuciones de GSAT como máximo a ejecutar. Corresponde a una aproximación de múltiple inicio. Existan gran cantidad de variantes de algoritmos para resolver una instancia de SAT mediante hill climbing³ y también utilizando otros algoritmos de búsqueda local⁴

3. Simulated annealing y el problema de corte máximo

Una importante crítica y limitación de los algoritmos ascenso de colinas es la imposibilidad de escapar de los máximos locales una vez llegados a los mismos. Una analogía entre el método y un escalador obtuso que asciende en la niebla se ha propuesto para explicarlo. En su visión limitada solo verá un radio limitado a su alrededor (los estados vecinos) y de forma codiciosa solo elige subir a la posición más elevada y nunca bajar. Claramente está preso de no lograr llegar a la cumbre por quedarse en picos inferiores. Para superar esta situación se han propuesto diferentes alternativas.

Un conjunto de estas son los llamados **algoritmos de umbral** (threshold algorithm). Estos corresponden a la familia de búsqueda local que eligen el próximo estado permitiendo seleccionar circunstancialmente una solución de menor valor de función costo. Evitan en general explorar todos los vecinos del estado actual para seleccionar de forma aleatoria un estado vecino como posible sucesor. Para esto definen una función vecindario de forma similar a Hill Climbing. Una vez elegido un estado vecino puede aceptarlo o rechazarlo como estado sucesor. Para esto tendrán en cuenta si es mejor al anterior o si no es peor

² Gent, I. and Walsh, T. 1992. The Enigma of SAT Hill-climbing Procedures. Research Paper 605, Dept. of AI, University of Edinburgh.

³ Ian P. Gent and Toby Walsh. 1993. Towards an understanding of hill-climbing procedures for SAT. In Proceedings of the eleventh national conference on Artificial intelligence (AAAI'93). AAAI Press, 28-33.

⁴ Hoos, Holger & Stützle, Thomas. (2000). Local Search Algorithms for SAT: An Empirical Evaluation. Journal of Automated Reasoning. 24. 421-481.

teniendo en cuenta un cierta diferencia (umbral) entre las funciones costo de los estados. De esa forma permiten empeorar hasta cierto límite. Este umbral suele ir modificándose por cada iteración. Permitiendo en las primeras iteraciones descensos más marcados y reduciendo progresivamente su valor hasta las últimas de ellas en donde solo se puede ascender (con el umbral de aceptación de peores estados equivalente a cero). En ciertas variantes incluso se pueden aceptar estados no factibles en las primeras iteraciones y rechazándolas en las últimas. Dado que los estados descubiertos posteriores pueden llegar a ser menores que alguno encontrado anteriormente se suele almacenar de forma separada la mejor solución encontrada hasta el momento independientemente al estado actual analizado.

Un ejemplo de algoritmo de umbral se conoce como **Threshold accepting**⁵ propuesto por Dueck y Scheuer. Este algoritmo selecciona un vecino e_v del estado actual e_a de forma aleatoria y compara sus funciones costo. Si el costo del nuevo estado supera al actual lo selecciona como el próximo. Si no lo supera, pero si su diferencia es menor a un valor “umbral” lo selecciona de todas maneras. En caso contrario, el estado actual se mantiene. Este proceso se repite iterativamente. En cada iteración el umbral se va reduciendo de forma determinística. En las primeras iteraciones es más factible seleccionar estados inferiores al actual. Luego, al aproximarse a la cantidad máxima de iteraciones a realizar el umbral disminuye hasta solo permitir mejorar para cambiar de estado. Los autores de este algoritmo presentaron también una versión totalmente determinista del mismo que llamaron **Deterministic Threshold Accepting**. En esta la elección del estado vecino candidato de cada iteración es también siguiendo una elección determinística.

El algoritmo de Threshold accepting fue una propuesta que se basó en un algoritmo de umbral anterior. Algoritmo que analizaremos a continuación y es conocido como **Simulated Annealing**⁶. Este nombre surge por analogía de un tratamiento térmico de ciertos metales en la metalúrgica: el recocido (annealing en inglés). El proceso busca un enfriamiento lento y controlado para reducir la dureza, aumentar la ductilidad del material trabajado previamente calentado.

⁵ G. DUECK, T. SCHEUER (1990). Threshold accepting: a general purpose optimization algorithm appearing superior to simulated annealing. Journal of Computational Physics 90, 161-175

⁶ S. KIRKPATRICK, CD. GELATT, JR., M.P. VECCHI (1983). Optimization by simulated annealing. Science 220, 671-680

Simulated Annealing es un algoritmo de umbral que utiliza una función vecindario para seleccionar de forma aleatoria el siguiente estado posible. Aceptará siempre modificar el estado si el vecino seleccionado es superior. De lo contrario utilizará una fórmula probabilística para decidir su aceptación. En la fórmula se trabaja con una variable nombrada como Temperatura "T". Esta variable se modificará progresivamente según avance la ejecución del algoritmo. Comienza con un valor inicial $T_0 > 0$. Cada iteración, este valor desciende de acuerdo a un **esquema de enfriamiento**. Propone la siguiente fórmula para determinar si un estado vecino e_v , que es inferior al estado actual e_a , se convierta en el nuevo estado según la función costo $f(e_v)$ y $f(e_a)$ respectivamente: $r < e^{(f(e_v) - f(e_a))/T}$, donde r es un número que se genera aleatoriamente entre 0 y 1. En pseudocódigo la aceptación o rechazo del siguiente estado corresponde a:

Selección de siguiente estado
Sea s el estado actual del problema Sea s_v el estado vecino candidato Sea T la temperatura Calcular c el valor de la función costo de s Calcular c_v el valor de la función costo de s_v Si $c_v > c$ Retornar s_v Calcular r de forma aleatoria entre 0 y 1. Calcular $\exp = (c_v - c) / T$ Si $r < (e \text{ elevado a } \exp)$ Retornar s_v Retornar s

Resta definir un esquema de enfriamiento. Hay diferentes alternativas. El más sencillo corresponde a multiplicar la temperatura T por una constante numérica " c " con un valor menor a 1 luego de cada iteración. La elección de esta constante determina la velocidad en la que se termina impidiendo seleccionar peores soluciones.

El siguiente es el pseudocódigo del algoritmo genérico:

Algoritmo genérico Simulated Annealing

Sea s_m el estado de función máxima encontrado al momento

Sea s el estado actual del problema

Sea Max la máxima cantidad de iteraciones

Sea $it=0$ la cantidad de iteraciones

Sea $T=T_0$ la temperatura

Sea c la constante de enfriamiento

Establecer en s y s_m el estado inicial

Repetir

 Seleccionar de forma aleatoria s_v un estado vecino de s

 Mediante T , s y s_v determinar el siguiente estado

 Establecer en s_m como el estado de costo máximo entre s_m y s

 Establecer $T = T * c$

 incrementar it en 1

Mientras que $it < Max$

Retornar s_m

Utilizaremos como ejemplo el problema del corte máximo. Recordemos su enunciado:

Problema del corte máximo

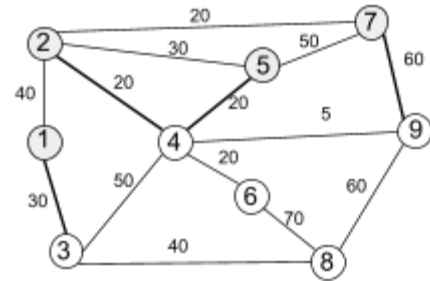
Dado un Grafo $G=(V,E)$ no dirigido y pesado. Queremos separar los nodos del mismo en dos conjuntos de tal forma de maximizar la suma de los pesos de los ejes que inician en un nodo de un conjunto y terminan en un nodo del otro conjunto.

Comenzamos definiendo como expresar una posible solución del problema. Consideramos que la cantidad de nodos en el grafo es n . Llamaremos "A" y "B" a los conjuntos donde cada nodo puede pertenecer. Enumeramos a cada uno de los nodos para identificarlos. Podemos representar una solución como un vector de " n " posiciones. El elemento de la posición i en el vector corresponde al nodo " i ". El valor en la posición del vector será 0 si el nodo se encuentra en el conjunto "A" o 1 si se encuentra en el conjunto "B". Existen 2^n posibles estados.

El estado inicial con el que comenzar la ejecución lo seleccionaremos de forma aleatoria entre un valor 0 y 2^n . Este valor expresado en binario corresponde a la asignación de los nodos a cada uno de los grupos.

Supongamos que la instancia del problema corresponde al grafo $G=(V,E)$ de 9 nodos que se muestra en la imagen.

Seleccionamos una asignación inicial aleatoria en dos conjuntos de cada uno de los nodos. Por ejemplo el siguiente vector $[0,0,1,1,0,1,0,1,1]$ corresponde a asignar a los nodos 1,2,5,7 en el conjunto "A" y 3,4,6,8,9 en el conjunto "B".



Con este corte podemos ver que los ejes que pasan entre los conjuntos corresponden a 1-3, 2-4, 5-4, 7-9 cuyos pesos son respectivamente 30,20,20 y 60. Podemos calcular entonces el valor del corte como 130, la suma de estos pesos. Inicialmente este es además el estado de mayor valor encontrado.

En segundo lugar debemos definir cuales corresponden a los estados vecinos de cada estado. En este caso seleccionamos aquellas asignaciones que se diferencian en 2 elementos. Es decir a la modificación de dos elementos diferentes de su conjunto actual. La elección del vecino candidato se realizará de forma aleatoria. Según la definición se deben seleccionar 2 nodos diferentes que cambiarán de conjunto. El siguiente pseudocódigo muestra el proceso de elección de un estado vecino candidato:

Obtener estado candidato vecino

Sea s el estado actual del problema

Sea n la cantidad de nodos

Sea s_v el estado candidato

Establecer $s_v = s$

Seleccionar un valor i entre 1 y n .

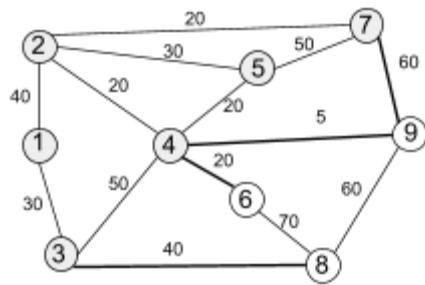
Establecer $s_v[i] = 1 - s_v[i]$

Seleccionar un valor j entre 1 y n (excepto i).

Establecer $s_v[j] = 1 - s_v[j]$

Retornar s_v

En cada iteración se evalúa el candidato seleccionado y se convierte en el próximo estado si mejora al actual o si supera la fórmula probabilística de umbral. Este proceso se repite tantas veces como se establezca en el algoritmo. El valor de temperatura se irá reduciendo. Generalmente requiere prueba y error establecer este valor y la constante "c" y otros para llegar a una buena solución.



Al seleccionar un estado vecino de forma aleatoria imaginemos que los nodos 3 y 4 son seleccionados para cambiar de conjunto. Se puede ver que los ejes en la frontera entre los subconjuntos cambian. En este candidato los ejes a sumar son 3-8, 4-6, 4-9 y 7-9. Los pesos de estos ejes adicionados dan como resultado 125. Es un valor menor al estado actual. Por lo tanto se debe utilizar la función probabilística que utiliza la temperatura y un valor aleatorio para determinar la aceptación o no del candidato como el próximo estado. El registro del estado analizado con mayor valor de corte no se modificará.

El siguiente pseudocódigo muestra el algoritmo para el problema de corte máximo:

Simulated Annealing: Corte Máximo

Sea s_m el estado de máximo corte encontrado al momento
 Sea s el estado actual del problema
 Sea max como la cantidad máxima de iteraciones a realizar

Sea $T=T_0$ la temperatura
 Sea c la constante de enfriamiento

Seleccionar de forma aleatoria una asignación inicial de s
 Establecer en s_m el estado s .

Definir $it=0$

Repetir

 Seleccionar de forma aleatoria s_v un estado vecino de s cambiando 2 nodos

 Mediante T , s y s_v determinar el siguiente estado

 Establecer en s_m como el estado de costo máximo entre s_m y s

 Establecer $T = T * c$

Incrementar it en 1
Hasta que $it > \max$

Retornar s_m

Para calcular la complejidad temporal del algoritmo debemos considerar los diferentes procedimientos realizados. El establecimiento del estado inicial de forma aleatoria se puede realizar en $O(n)$. Calcular el valor del corte se puede realizar en $O(m)$ que corresponde a la función de costo. “m” corresponde a la cantidad de ejes en el grafo. Buscar un vecino como candidato se puede realizar en $O(1)$ y Determinar el siguiente estado se realiza en $O(n+m)$. Todo el procedimiento se realiza una cantidad constante de veces. Por lo tanto se puede calcular la complejidad total del algoritmo como $O(m+n)$.

4. Tabu search y el problema de la asignación cuadrática

Con Simulated Annealing estudiamos una forma de escapar a los máximos locales en la búsqueda de la solución óptima. Para lograrlo se permitía, utilizando una variable aleatoria abandonar bajo ciertas condiciones un estado actual por uno alternativo de resultado inferior. La selección del estado alternativo se obtiene dentro de los estados definidos por la función vecindario. Esta elección también se realiza de forma aleatoria. A continuación veremos una diferente forma de evitar esta contrariedad: La conocida Como **Búsqueda Tabú** y propuesta por Fred Glover⁷ en su expresión moderna.

Para comenzar, es necesario explicar el significado de la palabra que le da el nombre a la metodología. **Tabú** significa “lo prohibido” en idioma polinesio. Corresponde a una creencia religiosa de pueblos de esta región que otorga el carácter de prohibido a ciertas acciones, lugares u objetos. En occidente esta palabra pasa a utilizarse en sociología para representar una conducta inmoral o inaceptable para un grupo de personas. Es desde este lugar donde el concepto es tomado prestado para elaborar una metodología heurística de búsqueda local. En este caso tabú corresponderá a la inhibición de realizar ciertas transiciones del estado actual a uno vecino teniendo en cuenta ciertas peculiaridades del

⁷ F. GLOVER (1986). Future paths for integer programming and links to artificial intelligence. Computers & Operations Research 13, 533-549

problema analizado y el historial de acciones realizadas durante una cierta cantidad de pasos anteriores.

El procedimiento utiliza un estado del problema como inicial. Ese estado tendrá un valor según la función costo. Comenzará un proceso iterativo en el que se deberán evaluar todos los posibles estados sucesores de acuerdo a una función de vecindad. Se debe seleccionar entre ellos, uno. En principio, al mejor entre todos, independientemente si supera o no al valor del corriente. Este pasará a ser el nuevo estado. Adicionalmente se contará con un registro del mejor estado hasta el momento encontrado. En cada iteración se debe verificar si el nuevo estado no supera a este y en caso afirmativo se actualizará el registro. En la búsqueda puede permitirse ingresar a estados que no cumplan con algunas de las restricciones del problema (es decir que no sean estados factibles). Sin embargo al registrar el mejor estado encontrado hasta el momento todas las restricciones deben tenerse en cuenta.

Se puede observar que, seleccionar un próximo estado independientemente si supera al estado actual determina una manera de escapar a un máximo local. Sin embargo, también nos lleva a un nuevo problema. En el nuevo estado, en la próxima iteración, nada impide que regresemos al estado previamente abandonado. Podemos sucumbir a un ciclo sin fin que nos impida explorar otros estados del problema. Recordar el estado anterior, no resuelve el problema. Podemos experimentar en la exploración ciclos de diversos tamaños que nos atrapen. Un mecanismo de inhibición de estados previos, por lo tanto, se hace deseable. Este, puede tomar diferentes formas de acuerdo al tipo de problema a resolver. Por lo general se establece que durante una cierta cantidad de iteraciones cierto movimiento o modificaciones no se pueden realizar. Esto conforma una lista de **movimientos tabú**. Lo que finalmente le brinda el nombre a la metodología.

Veamos algunas opciones de manejo de registro de movimientos tabú. Estos se relacionan íntimamente con el tipo de problema y la manera de representar los estados del problema. Una representación común a diversos problemas corresponde a una elección de un subconjunto de elementos dentro de un conjunto. Para ello podemos utilizar un vector binario. La posición i en 1 representa la selección del elemento i en la solución. Mientras que esta misma posición en 0 representa lo contrario. Un cambio de estado a uno vecino se podría construir modificando una o varias de estas posiciones alternando su valor. El

movimiento por lo tanto corresponde a esta alteración. Una estrategia para el tabú es simplemente registrar cuál o cuáles de las posiciones fueron modificadas y no permitir volver a hacerlo por una cantidad próxima de iteraciones. Para eso podemos tener una estructura de movimientos tabú donde lo registramos.

En otros tipos de problemas la solución se puede establecer como una permutación de un conjunto de elementos. En este caso los movimientos para transformar los estados - que están íntimamente relacionados con la naturaleza de la función vecindad elegida - pueden ser intercambiar un par de elementos, seleccionar un elemento y llevarlo a una posición o alguna otra variante. En ese caso el movimiento tabú puede ser evitar que un determinado elemento (una vez seleccionado) se vuelva a permutar durante un número de transiciones, un par de elementos no puedan ser permutados entre sí. O un determinado elemento no pueda volver a una determinada posición. De acuerdo a la naturaleza de la restricción seleccionada será la evolución del dominio de la función de vecindad. Pueden ser reglas muy estrictas que acoten rápidamente la variedad de movimientos o más laxas. En el primer caso es más probable recorrer estados más distantes, por lo contrario es más probable volver a visitar estados previamente analizados.

Regresamos al proceso de elección del próximo estado en el proceso iterativo. Al explorar la función de vecindad se debe evitar considerar a aquellos estados en los que para alcanzarlos se debe realizar algún movimiento tabú. Este cambio nos permite hablar de una función de vecindad dinámica cuyo estados accesibles se modifican de acuerdo al estado actual y el historial reciente. Al evaluar los estados vecinos se seleccionará como estado próximo a aquel estado vecino que brinde la mejor solución y que no requiera realizar algún movimiento tabú para llegar a este.

Consideramos un problema genérico e imaginemos que en un determinado momento al analizar los estados vecinos el primer o primeros mejores resultados se pueden conseguir mediante algún movimiento tabú. Según lo establecido estos se deberían descartar. Pero supongamos que al menos uno de ellos corresponde a una solución factible que supera a la mejor encontrada hasta ahora. No parece ser razonable realizarlo. Llamaremos **condiciones del nivel de aspiración** (aspiration level conditions) a los criterios para los que la prohibición de un determinado movimiento tabú es anulada. El caso mencionado corresponde al uso más extendido de esta práctica.

Es crucial aclarar que las reglas de movimientos tabú intentan evitar regresar a un estado previamente visitado, aunque al buscarlo es posible de forma no esperada restringir estados inexplorados. Por eso la importancia de los criterios de aspiración.

Existen dos mecanismos adicionales que se suelen aplicar en esta metodología: La intensificación y la diversificación. Estas impactan a la hora de determinar el siguiente estado y tienen el comportamiento opuesto entre sí. La **intensificación** corresponde al procedimiento de favorecer la selección de movimientos previamente realizados que brindaron buenos resultados en la búsqueda del máximo. Se busca repetir ciertas configuraciones o elementos exitosos. La **diversificación** persigue favorecer la realización de movimientos poco o nunca visitados para permitir explorar regiones del espacio de estados poco analizados. Ambos se consiguen mediante la modificación de la función costo incluyendo modificadores positivos y/o negativos que impactan en el cálculo de cada estado.

La aplicación de la intensificación y diversificación se presenta de diversas formas. Sus versiones más simples implican contar la cantidad de veces que cada movimiento es seleccionado y un indicador relativo a los incrementos (o decrementos) logrados en su elección. Estos valores normalizados se aplican al cálculo de la función costo.

Con todo los conceptos establecidos, es momento de presentar en pseudocódigo la estructura genérica de un algoritmo de búsqueda Tabú:

Algoritmo genérico de Búsqueda Tabú

Sea s_m el estado de función máxima encontrado al momento Sea s el estado actual del problema Sea Max la máxima cantidad de iteraciones Sea $it=0$ la cantidad de iteraciones Seleccionar de forma aleatoria una asignación factible inicial de s Establecer en s_m el estado s . Calcular c el valor de la función de costo de s_m Definir $it=0$ Repetir Buscar s_v el estado vecino de s con mayor función costo c_v y teniendo en cuenta la
--

tabla tabú, los criterios de aspiración, la intensificación y diversificación.

Si $c_v > c$ y s_v es una solución factible

Definir s_m como s_v .

Definir c como c_v .

Definir s como s_v

Actualizar la tabla tabú e información para la intensificación y diversificación.

incrementar it en 1

Mientras que $it < \text{Max}$

Retornar s_m

Utilizaremos como ejemplo el problema de asignación cuadrática. Establezcamos su enunciado:

Problema de la asignación cuadrática

Dadas “n” ubicaciones cuya distancia entre cada par de ellas x,y conocemos como $b_{x,y}$. “n” plantas distribuidoras de productos cuyo flujo diario entre cada par de ellas w,z conocemos como $a_{w,z}$. Queremos determinar en qué ubicación establecer cada planta de forma tal de minimizar la sumatoria de las distancias de transportes de los productos. Mediremos esta cantidad como $\sum_{i=1}^n \sum_{j=1}^n a_{i,j} b_{\pi(i),\pi(j)}$ con $\pi(x)$ la asignación de la planta “x” a una determinada ubicación.

Podemos pensar una asignación de plantas distribuidoras a ubicaciones como un vector π de “n” posiciones. La posición i corresponde a asignar una planta determinada a la ubicación i . Un valor numérico entre 1 y n en cada posición, el identificador de la planta, se asigna. Podemos calcular todas las soluciones posibles como las permutaciones de los “n” valores.

Supongamos que tenemos 10 ubicaciones y plantas de distribución. Contaremos con una matriz A de 10x10 que nos permite conocer el flujo entre cada par de plantas. Además una matriz B del mismo tamaño que nos permite conocer la distancia entre cada par de ubicaciones. Un vector de 10 posiciones representa una posible configuración. Por ejemplo [2,4,8,1,7,10,3,9,5,6] corresponde a ubicar a la planta 2 en la ubicación 1, la planta 4 en la ubicación 2 y

progresivamente para el resto de las plantas. Existen $10!$ posibles permutaciones (3628800 posibles soluciones). En este problema todas las permutaciones son posibles soluciones factibles.

La función vecindad estará definida por todos aquellos estados accesibles realizando un intercambio de lugar entre dos plantas. Consideraremos a este intercambio como un movimiento que permite acceder a un estado vecino. Al tener “n” posiciones, existen $n*(n-1)/2$ posibles movimientos.

Nos encontramos actualmente en el estado [2,4,8,1,7,10,3,9,5,6] y consideramos como posibles movimientos el intercambio de 2 centros de distribución. Un posible movimiento corresponde a cambiar el centro 8 por el 5. Esto dará como resultado el estado [2,4,5,1,7,10,3,9,8,6]. Existen 45 posibles intercambios y por lo tanto esa es la cantidad de vecinos entre cada estado.

Al momento de evaluar la vecindad y elegir un próximo estado se habrá seleccionado un movimiento de los disponibles. Ese se convertirá en un movimiento tabú por una cantidad posterior de “t” iteraciones. Solo podrá ser seleccionado si al hacerlo la evaluación de la función costo es mejor a la de la mejor solución encontrada hasta el momento (condiciones del nivel de aspiración).

Si establecemos que solo luego de $t=3$ movimientos es posible seleccionar nuevamente uno marcado como tabú. Entonces de forma simultánea esa será la máxima cantidad de intercambios prohibidos. Supongamos que nos encontramos en la iteración 15 en la que nos encontramos en el estado [2,4,8,1,7,10,3,9,5,6] y desde el estado inicial realizamos los movimientos indicados en la tabla que acompaña este párrafo. Los movimientos 14,13 y 12 corresponden a los tabú. No se puede seleccionar a menos que mejoren el mejor resultado obtenido (criterio de aspiración). Se puede observar además que algunos de los ya se realizaron anteriormente. Por ejemplo el movimiento (2,5) se realizó en las iteraciones 2,8 y 14. Cada uno de esos estados está a una distancia mayor a 3 entre sí.

1	(6,8)
2	(2,5)
3	(6,7)
4	(4,6)
5	(3,7)
6	(2,10)
7	(6,7)
8	(2,5)
9	(3,7)
10	(2,10)
11	(1,8)
12	(7,10)
13	(4,9)
14	(2,5)

Al seleccionar el movimiento se incrementará un contador asociado al mismo. De esa forma podemos conocer cuales son los movimientos más seleccionados y cuáles menos. Esto nos servirá para aplicar la diversificación: aquellos movimientos más visitados serán

penalizados. La penalización corresponderá a un decremento en la función costo del estado resultante de aplicar ese movimiento.

	1	2	3	4	5	6	7	8	9	10
1								11		
2					14					10
3							9			
4						4			13	
5		3								
6				1			7	1		
7			2			2				12
8	1					1				
9				1						
10		2					1			

Representaremos los movimientos realizados mediante una matriz. En el triángulo superior derecho cada posición (x,y) representa a realizar el movimiento de intercambiar el elemento x con el y (o el y con el x que corresponde al mismo resultado). El contenido de la celda corresponderá a la iteración donde por última vez se realizó ese intercambio (o vacío si nunca ocurrió). Un movimiento tabú simplemente se calcula restando el número de iteración actual con la iteración registrada en la celda. En el triángulo inferior izquierdo cada posición (y,x) representa la cantidad de veces que se realizó el movimiento de intercambiar el elemento “ x ” con el “ y ”. Por ejemplo el movimiento $(2,5)$ observando en la celda 5,2 podemos asegurar que fue realizado un total de 3 veces. Podemos utilizar la cantidad registrada para cada movimiento dividido la cantidad de iteraciones realizadas como un porcentaje sobre el total de movimientos realizados. Podemos utilizarlo como modificador a la función costo para intentar diversificar la elección. A mayor porcentaje, mayor penalidad.

Finalmente para aplicar intensificación registramos, cada vez que seleccionamos un movimiento dos valores: el máximo y mínimo incremento obtenido en su aplicación. De esta manera podemos hacer un promedio entre estos valores para determinar aquellos movimientos más efectivos. Aquellos movimientos más efectivos serán recompensados en la función costo al aplicar el movimiento.

Cada vez que seleccionamos un movimiento podemos calcular la diferencia con el estado anterior (sin los modificadores por intensificación ni diversificación) y ver cuánto incrementa o decrementa el valor del estado actual. Podemos almacenar el máximo incremento histórico y el mínimo. Este incremento mediante un escalar lo podemos adicionar al cálculo de la función costo. De esa forma aquellos estados que históricamente nos brindaron un mejor resultado son ponderados en la elección de los próximos estados.

Para la búsqueda tabú se comenzará por un estado inicial. Este tendrá su valor costo asociado. La función costo que utilizaremos es la brindada por el enunciado. A esta adicionamos los modificadores por intensificación y diversificación según definimos

anteriormente. Se evalúan todos los estados vecinos y se selecciona el que brinde mayor función costo teniendo en cuenta los modificadores por intensificación y diversificación. Se excluyen los estados a los que se debe acceder a través de un movimiento tabú (excepto que supere el criterio de aspiración). Una vez seleccionado el próximo estado se debe verificar si supera al mejor estado conocido hasta el momento y en caso afirmativo reemplazarlo. Puede ocurrir, dependiendo el problema que el estado encontrado no corresponda a una solución factible al no cumplir una restricción del problema. En ese caso la actualización no se debe llevar adelante. Para este problema toda solución es factible. Para finalizar la iteración se debe actualizar las tablas con la información de los estados tabú. El proceso se repite una cantidad preestablecida de iteraciones. Al finalizar se retorna al mejor estado encontrado.

A continuación se presenta el pseudocódigo específico para resolver este problema. En primer lugar se presenta el cálculo la función costo (sin modificadores):

Cálculo de función costo: Problema de la asignación cuadrática

<pre>Sea as la asignación a analizar del problema Sea b[i,j] la distancia entre la ubicación "i" y la "j" Sea a[x,y] el flujo diario entre la planta "x" e "y" Sea costo el costo acumulado costo = 0 Desde i = 1 a n Sea x = s[i] Desde j = 1 a n si i<>j Sea y = s[j] costo += a[i,j] * b[x,y] Retornar costo</pre>

Del análisis del pseudocódigo se puede observar que la complejidad espacial es $O(1)$ y la complejidad temporal es $O(n^2)$. Consideramos que s es un vector; a y b matrices de $n \times n$.

A continuación vemos las funciones que obtienen y actualizan los modificadores para la diversificación e intensificación. Además que ayudan a la determinación de la próxima asignación.

Manejo de diversificación e intensificación: Problema de la asignación cuadrática

Sea M la matriz de movimientos

Sea T la matriz de incrementos

Sea it el número de iteración actual

Sea M_{ij} el movimiento de intercambiar la planta i con la j

Sea $incr$ el incremento obtenido luego de aplicar el movimiento M_{ij}

Obtener valores de intensificación y diversificación:

Sea $t=3$ la cantidad de iteraciones que un movimiento es tabu

Sea es_tabu el indicador si el movimiento M_{ij} lo es

Si $it - M[i,j] \leq t$ entonces

$es_tabu = 'Si'$

sino

$es_tabu = 'No'$

Sea $\#sel = M[j,i]$ la cantidad de veces que el movimiento M_{ij} fue seleccionado

Sea $maxIncr = T[j,i]$ el máximo incremento obtenido al realizar el movimiento M_{ij}

Sea $minIncr = T[i,j]$ el mínimo incremento obtenido al realizar el movimiento M_{ij}

Retornar $es_tabu, \#sel, maxIncr, minIncr$

Actualizar valor de intensificación y diversificación:

$M[i,j] = it$

Incrementar $M[j,i]$ en 1

Si $incr > T[j,i]$

$T[j,i] = incr$

Si $incr < T[i,j]$

$T[i,j] = incr$

Todas las operaciones realizadas son $O(1)$ en ambas funciones. Se utiliza un $O(n^2)$ espacial para almacenar las matrices de movimientos e incrementos.

Con estas funciones ya podemos generar la parte central del algoritmo, la selección de la próxima asignación

Selección de la próxima asignación: Problema de la asignación cuadrática

Sea as_{min} la asignación de mínima función corte encontrado al momento globalmente

Sea $fc_{as_{min}}$ el cálculo de la función costo de as_{min}

Sea as la asignación actual del problema

Sea fc_{as_m} el cálculo de la función costo de as_m

Sea as_m la mejor asignación candidata encontrada en la iteración

Sea M_{ij} el movimiento con el que se obtiene la mejor asignación en la iteración

Sea $fmod_{as_m}$ el valor de la mejor asignación candidata encontrada en la iteración

Establecer $fmod_{as_m} = +\infty$

Desde $i = 1$ a n

 Desde $j = i+1$ a n

 Sea M_{ij} el movimiento de intercambiar la planta i con la j

 Sea as_c la asignación resultante de aplicar M_{ij} a as .

 Obtener valores de intensificación y diversificación para M_{ij}

 Sea fc_{as_c} el cálculo de la función costo de as_c

 Sea $fmod_{as_c}$ el cálculo de aplicar los intensificación y diversificación a fc_{as_c}

 Si $fmod_{as_c}$ es superior a $fmod_{as_m}$

 Si $es_tabu = 'No'$ o fc_{as_c} es superior a $fc_{as_{min}}$

 Establecer a $as_m = as_c$

 Establecer $M_{ij} = M_{ij}$

Actualizar valor de intensificación y diversificación de M_{ij}

Retornar as_m

La complejidad de esta función está dada por la doble iteración y los cálculos dentro de ella. La mayor complejidad es la función del cálculo de la función costo: $O(n^2)$. Se realiza un total de $O(n^2)$ veces. Por lo tanto la complejidad total de esta función se puede estimar como $O(n^4)$. La complejidad espacial requiere almacenar asignaciones y por lo tanto es $O(n)$.

Finalmente, unificando las funciones anteriores, podemos concluir la metodología iterando una cantidad predeterminada de veces:

Búsqueda Tabú: Problema de la asignación cuadrática

Sea as la asignación actual del problema

Sea as_{min} la asignación de mínima función corte encontrado al momento

Sea max como la cantidad máxima de iteraciones a realizar

Seleccionar de forma aleatoria una asignación inicial as

```
Establecer en  $as_{min}$  la asignación  $as$ .  
Sea  $fc_{as_{min}}$  el cálculo de la función costo de  $as_{min}$ 
```

```
Definir  $it=0$ 
```

```
Repetir
```

```
    seleccionar  $as_m$  la próxima asignación partiendo de  $as$ 
```

```
    Sea  $fc_{as_m}$  el cálculo de la función costo de  $as_m$ 
```

```
    Si  $fc_{as_{min}} > fc_{as_m}$ 
```

```
         $as_{min} = as_m$ 
```

```
         $fc_{as_{min}} = fc_{as_m}$ 
```

```
    Establecer en  $as_m$  la asignación  $as_m$ .
```

```
    incrementar  $it$  en 1
```

```
Hasta que  $it > max$ 
```

```
Retornar  $as_{min}$ 
```

La complejidad final es la suma de la complejidad de seleccionar la próxima asignación y la del cálculo de la función costo. Esto multiplicada por la cantidad de iteración. Esto último al ser un número constante hace que en su expresión de complejidad se excluya. Podemos entonces determinar la complejidad temporal del algoritmo como $O(n^4)$.

Es posible reducir los tiempos de ejecución realizando algunos de los pasos de forma más eficiente. Pero a efectos de la explicación del ejemplo se obvian para tratar de dejar el procedimiento lo más claro posible.

5 Algoritmos genéticos y el problema de las 8 reinas

Las anteriores metodologías presentadas comienzan desde un estado solución del problema y realizan una selección entre estados vecinos del próximo. Realizan esta operación iterativamente hasta finalizar. Se puede realizar sobre ellas un seguimiento lineal entre el estado inicial y el final. Uno de ellos será el estado de mayor valor obtenido (no necesariamente el óptimo). Se pueden ejecutar de forma repetida para permitir explorar diferentes caminos y tal vez encontrar soluciones mejores. Cada ejecución es independiente de las anteriores y no puede aprovechar lo aprendido o el espacio explorado. Existen diversas propuestas para intentar capitalizar los conocimientos

anteriores. A continuación analizaremos uno de ellos que toma como inspiración los procesos evolutivos de la vida en nuestro planeta.

Se conoce como **algoritmos genéticos** a una rama dentro de los algoritmos evolutivos. Estos toman conceptos de la selección natural en biología en el que los individuos más aptos producen mayor descendencia. Dentro de los **algoritmos evolutivos** también se encuentran la programación genética (genetic programming) y estrategias de evolución (Evolution Strategies). Los primeros trabajos en el estudio de los algoritmos genéticos para la resolución de problemas son acreditados a John Holland^{8 9} a fines de la década del 60 y principios de la siguiente en la universidad de Michigan. Las definiciones de Holland se basan en trabajos anteriores sobre biología, genética y la teoría de la evolución de Darwin.

El algoritmo comienza con una **población** de “p” individuos. Cada **individuo** corresponde a la representación de un estado solución posible del problema a resolver. Estos se pueden construir de forma aleatoria o pueden ser candidatos obtenidos mediante la ejecución de otros algoritmos. Un algoritmo genético se ejecuta una cantidad determinada de veces de forma iterativa. Cada iteración se conoce como una **generación** y estará conformada por una población que es descendiente de la población de la generación anterior. El tamaño de la población se puede mantener entre generaciones o puede modificarse.

Siguiendo el modelo biológico cada individuo internamente contiene **cromosomas** (chromosomes) que expresan su naturaleza interna. Este no es más que un vector con el estado de solución. John Holland utiliza en su presentación del método un vector binario. Donde cada bit corresponde a una variable del problema y recibe el nombre de **gen** (gene) y su posible valor un **alelo** (alleles). Posteriormente se trabajó en la representación del vector con variables más complejas como números enteros o reales. Cualquiera de las representaciones de estados solución que trabajamos en apartados anteriores se pueden utilizar como cromosomas. En esos casos las variables (el gen) son más complejas y existen más alelos para los mismos. En ciertos problemas la posición del gen modifica el estado de solución. Se conoce como **locus** a justamente la posición del gen dentro del cromosoma.

⁸ Holland, John H.. “Adaptation in natural and artificial systems.” (1975).

⁹ Holland, John H.. “Genetic Algorithms Computer programs that " evolve " in ways that resemble natural selection can solve complex problems even their creators do not fully understand.” (2005).

Veamos algunos ejemplos simples. En el problema de la mochila queremos maximizar la ganancia al seleccionar ciertos elementos con valor cuyo peso combinado no puede superar la capacidad de la mochila. Si tenemos “n” elementos podemos representar el cromosoma de un individuo como un vector binario de “n” posiciones. Cada gen corresponde a un elemento que puede estar en la mochila. Los alelos son 0 (no) y 1 (sí) y determinan si el elemento se encuentra seleccionado en la solución. En el problema de coloreo de grafos queremos ver si podemos etiquetar con “k” valores diferentes los nodos de un grafo de modo que no hayan nodos adyacentes con la misma etiqueta. Si el grafo tiene “v” vértices, podemos representar el cromosoma de un individuo como un vector de números enteros, donde cada gen corresponde a un nodo del grafo y sus alelos pueden ser alguna de las “k” etiquetas posibles. Finalmente, en el problema del viajante de comercio queremos minimizar el recorrido de una persona a través de un conjunto de “c” ciudades realizando un circuito donde cada una de ellas se puede visitar una única vez. Un cromosoma será un vector donde cada gen corresponde a la representación de una ciudad. En este caso cada gen puede ubicarse una vez en una posición del cromosoma. El orden de los genes determina el orden del circuito.

El concepto de esquema (schema) es clave dentro de los algoritmos genéticos y para entender su éxito en conseguir buenos resultados. Se conoce como **esquema** a un conjunto de cromosomas que tienen un patrón común entre ellos. Podemos pensarlo como el conjunto de individuos posibles que comparten ciertos genes en común. Lo podemos representar como el vector con ciertos valores fijos y otros no determinados.

Se espera que individuos “exitosos” puedan tener esquemas internos beneficiosos. Estos serán más proclives a dejar descendencia en la siguiente generación que compartan estos esquemas. La medición del “éxito” de un individuo (y por carácter transitivo un cromosoma en particular) se realiza mediante una **función de aptitud** (fitness function). En un problema de optimización esta puede ser la función costo de la misma. Aunque existen propuestas de modificadores para ciertos problemas en esta función.

Para la determinación de la próxima generación se aplican **operadores genéticos** (Genetic Operators) sobre la población. Se han propuesto gran cantidad de operadores, los más habituales corresponden a selección (selection), recombinación (recombination) y mutación (mutation).

La **selección** opera sobre la totalidad de los individuos de la población. Determina cuáles de ellos “sobreviven” para generar descendencia en la siguiente generación. Habitualmente se utiliza para eso la función de aptitud. Puede determinarse un porcentaje del total de la población realizando un corte según mayor **valor de aptitud** (score fitness) calculado mediante la función de aptitud. Otras alternativas podrían ser realizar un corte según un determinado valor de aptitud o probabilísticamente también utilizando este valor (por ejemplo dividiendo el valor de aptitud del elemento por la sumatoria del valor de aptitud de toda la población).

La **recombinación** corresponde a generar la nueva generación en base a los individuos que pasaron la selección. El proceso de recombinación más sencilla posible es simplemente tomar cada individuo y crear uno nuevo con su mismo cromosomas. En caso de hacer falta crear nuevos individuos para llegar a un valor determinado de población se puede seleccionar de forma aleatoria a alguno de ellos para copiarlo más de una vez utilizando el valor de aptitud en la función probabilística. Se puede ver este proceso como una reproducción asexual donde los descendientes mantienen las mismas características que su progenitor. En este caso el siguiente operador llamado “mutación” resulta clave para el buen funcionamiento del algoritmo. Otra alternativa y posiblemente la más utilizada corresponde a seleccionar una pareja de individuos y generar descendientes combinando los cromosomas de estos. No es habitual pero el emparejamiento se puede dar también entre 3 o más individuos

Profundizaremos sobre el emparejamiento entre dos individuos. Existen diferentes estrategias para realizarlo. En primer lugar se debe determinar cómo armar las parejas. La opción más sencilla correspondierealizar un pareo al azar. Se puede o no privilegiar a aquellos individuos de mayor valor de aptitud. Holland propuso tomar de forma fija a los individuos con mayor valor de aptitud y a estos asignarles una pareja de forma aleatoria.

Una vez formadas las parejas se debe realizar el proceso de cruce. Una opción habitual es seleccionar de forma aleatoria una posición “i” del vector del cromosoma. El nuevo individuo se conforma con los primeros “i” genes del primer padre y los restantes obtenidos del segundo padre desde el gen i+1 al final. De forma equivalente se puede armar un segundo descendiente con los genes descartados del primer cruzamiento. Otra opción es por cada gen determinar aleatoriamente de cuál de los padres se seleccionará.

Estos procesos se pueden extender a cruzamiento con 3 o más progenitores sin dificultad alguna. Si el problema a resolver depende de la ubicación del gen (donde la solución representa una secuencia por ejemplo), estos modos de cruce no son utilizables. En su lugar se puede determinar una o más ubicaciones en el vector. Por cada uno ver que genes se encuentra en los padres y eso transformarlo en un intercambio de posiciones entre ellos en sus descendientes.

Finalmente la **mutación** corresponde a la modificación de forma aleatoria de los genes de los individuos. Se establece un valor umbral entre 0 y 1 que corresponde a la probabilidad de que un gen modifique su valor. Este valor puede ser fijo o ir disminuyendo por cada generación. Por cada gen de cada individuo se determina si se produce una mutación. La mutación del gen puede ser una modificación del alelo o locus según el problema lo requiera. Este procedimiento enriquece la variedad cromosómica de la población y ayuda a explorar diferentes estados de la solución.

El conjunto de los descendientes construidos forman parte de la nueva generación. En algunas propuestas se agregan los mejores individuos de la generación anterior. Este proceso se conoce como **elitismo** (elitism). Con esto lograr que entre generación se mantenga el mejor resultado obtenido hasta el momento.

Luego de aplicar los operadores genéticos ya se cuenta con una nueva generación y se puede volver a repetir iterativamente el proceso. Al finalizar se obtiene el individuo más apto y el estado solución del problema que representa como resultado del algoritmo.

Veamos a continuación el pseudocódigo del proceso general. Este puede modificarse según decisiones de implementación este podría verse modificado ligeramente

Algoritmo genético genérico
Sea P la población de individuos Sea max como la cantidad máxima de generaciones a realizar Establecer P con una población inicial de "p" individuos Definir it=0 Repetir Por cada individuo i de P calcular su función de aptitud $f_a(i)$

Crear S el subconjunto P que superan el criterio de selección
Crear E el emparejamiento resultando de S
Crear H los descendientes de las parejas E
Aplicar a cada individuo de H la operación de mutación
Construir P' la nueva generación con H y una elección por elitismo de S.

incrementar it en 1
Establecer P con P'

Hasta que $it > max$

Retornar estado solución del individuo p de P con mejor función costo

A continuación realizaremos un ejemplo de este algoritmo en el problema de las 8 reinas.
Recordemos su enunciado:

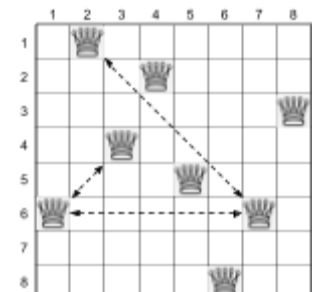
Problema de las 8 reinas

En un tablero de ajedrez se quieren ubicar 8 reinas de tal forma que ninguna de ellas se ataquen entre sí.

Este clásico problema no es de optimización, pero se puede modificar levemente para poder convertirlo. Pediremos maximizar la suma de reinas en el tablero para que no se ataquen entre sí. También lo podríamos expresar como minimizar la cantidad de ataques entre las piezas. Recordamos que esta pieza ataca a todos los que se ubiquen en su misma fila, columna o diagonales.

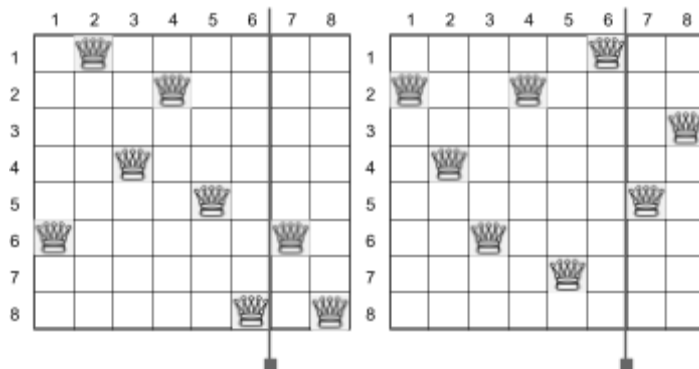
Podemos representar una posible solución mediante un vector de 8 posiciones. Cada posición corresponde a una columna (evitamos poner más de una reina por columna para que no se ataquen entre sí) y en cada celda podemos incluir un número entre 1 y 8 que corresponde al número de fila donde incluimos a la reina en esa columna. Este vector corresponde al cromosoma de un individuo que tiene 8 genes con 8 alelos posibles.

En la imagen que acompaña al párrafo se puede ver una posible solución al problema de las 8 reinas que no es óptima. Se observa que 4 reinas se atacan entre sí. Por lo tanto el número de reinas que no se atacan es 4. Podemos expresarlo como un individuo cuyos cromosomas los expresamos como (6,1,4,2,5,8,6,3).



Plantearémos una población inicial de 100 individuos. Por cada individuo tendremos su cromosoma. Establecerémos de forma aleatoria a cada individuo por primera vez. Esa será la generación cero o inicial. Definimos la función de aptitud como la suma de reinas en el tablero para que no se ataquen entre sí.

La selección corresponderá a los 80 individuos con mayor valor de aptitud. Crearemos parejas entre ellos de forma aleatoria de forma que cada uno se encuentre con una pareja diferente. Para lograrlo realizamos una mezcla aleatoria y las parejas serán los individuos contiguos. Por cada pareja realizaremos una elección aleatoria entre 1 y 7. El resultado será el punto de cruce entre la cadena de los pareja donde formaremos dos descendientes. El primero con la primera parte del primer progenitor sumada a la segunda parte del segundo progenitor. El segundo hijo con las cadenas no utilizadas en el primero. De esa forma se crearán 80 descendientes.



Consideremos la pareja A: (6,1,4,2,5,8,6,8) y B: (2,4,6,2,7,1,5,3). Al momento de realizar la recombinación al azar obtenemos el valor 6. Por lo tanto cada individuo partirá su cadena en dos. La primera parte con los 6 primeros genes y la segunda con los 2 siguientes.

Tendremos las siguientes subcadenas A1: (6,1,4,2,5,8) y A2:(6,8) y B1:(2,4,6,2,7,1) B2:(5,3). Con ellos crearemos dos descendientes: C como A1 unión B2 y D como B1 unión A2. Respectivamente (6,1,4,2,5,8,5,3) y (2,4,6,2,7,1,6,8).

La mutación utilizará como umbral de cambio un 5% de probabilidad. Por cada uno de los hijos generados y por cada uno de sus genes, se realiza una evaluación aleatoria para determinar si se produce una mutación. En caso positivo una tirada aleatoria determina que valor entre 1 y 8 tomará ese gen.

Finalmente para completar la nueva generación y se mantenga en 100 la población, se seleccionan los 20 individuos de mayor para que formen parte de la siguiente. El proceso se vuelve a repetir hasta una cantidad determinada de veces. En la última generación se

analiza cuál es el individuo de mayor valor costo (cantidad de reinas que no se atacan). En este caso equivale a la función de aptitud. Se retorna como resultado del algoritmo el mejor individuo encontrado.

El siguiente es el pseudocódigo de la solución propuesta:

Problema de las 8 reinas: Algoritmo genético

Sea P la población de individuos

Sea max como la cantidad máxima de generaciones a realizar
--

Desde $i=1$ a 100

Desde $j=1$ a 8

$P[i][j]$ = valor aleatorio entre 1 y 8

Definir $it=0$

Repetir

Sea P' la población de la próxima generación
--

Sea E la población de la generación actual de mayor valor de aptitud
--

//Selección

Por cada individuo i de P calcular su función de aptitud $f_a(i)$

Ordenar P por su función de aptitud.

Incluir en E los primeros 20 individuos de P.

Eliminar los últimos 20 individuos de P

//Combinación

Mezclar aleatoriamente los individuos de P
--

Desde $m=1$ a 40 sumando de a 2

Sea $a = P[m]$

Sea $b = P[m+1]$

Sean c, d individuos descendientes

Establecer cruce un valor aleatorio entre 1 y 7

Desde $j=1$ a cruce

$c[j]=a[j]$

$d[j]=b[j]$

Desde $i=cruce+1$ a 8

$c[j]=b[j]$

$d[i]=a[i]$

```

        Agregar en P' a c y d.

//Mutación
Desde m=1 a 80
    Desde j=1 a 8
        Calcular r un valor aleatorio entre 1 y 100.
        Si r<=5
            establecer P'[i][j] con un valor aleatorio entre 1 y 8

//Elitismo
Agregar E en P'
Establecer P con P'

incrementar it en 1

Hasta que it > max

Obtener de P el resultado con mayor función costo y retornarlo

```

La complejidad espacial del algoritmo es en función del tamaño de la población y del cromosoma del individuo. El primero está planteado como una constante de tamaño 100. Sin embargo para hacerlo más genérico podemos llamarlo p y que en esta instancia particular toma ese valor. El segundo tiene un valor de 8. De forma genérica indicaremos que tiene un tamaño “ n ” (podemos plantear el problema para tableros más grandes). Necesitamos para almacenar toda la población $O(n \cdot p)$ de espacio.

En cuanto a la complejidad temporal, podemos considerar la cantidad de iteraciones una constante. El cálculo de la función de aptitud se puede realizar en $O(n^2)$. El mismo se realiza “ p ” veces por generación. Es decir que su complejidad se puede estimar en $O(p \cdot n^2)$. Ordenar la población se puede realizar en $O(p \log p)$. La combinación y la mutación se puede estimar en $O(p \cdot n)$. La verificación final corresponde a evaluar la última generación para encontrar el individuo más apto en $O(p \cdot n^2)$. La complejidad total es la suma de estas complejidades individuales.

Antes de finalizar veamos una variante. Se puede evitar crear individuos que a priori sabemos que no serán óptimos. En la definición del estado del problema estamos admitiendo que varias reinas se encuentren en la misma fila. Esto se puede evitar restringiendo que los valores de las variables no pueden utilizar los previamente

establecidos. Esto nos lleva a representar un estado del problema como una permutación de reinas en diferentes filas. Con esta representación no podemos usar el mismo operador de recombinación y mutación. Para resolverlo debemos cambiarlo.

Para la combinación podría ser obtener una posición de los vectores padres y ver que gen tienen cada uno de ellos. Luego intercambiar el par dentro de cada cromosoma. Esto se puede repetir para varias posiciones. La mutación podría ser la probabilidad de rotar un subvector desde esa posición hasta el final dentro del individuo.

Consideremos la pareja A: (6,1,4,2,5,3,7,8) y B: (1,3,6,4,8,2,5,7). Al momento de realizar la recombinación al azar obtenemos el valor 6. Vemos que en la posición 6 tenemos en A el valor 3 y en "B" el valor 2. Intercambiaremos entonces en cada uno de los individuos la posición del gen con valor 3 y 2 en el vector. Quedará de la siguiente manera los descendientes: C: (6,1,4,3,5,2,7,8) y D: (1,2,6,4,8,3,5,7). Supongamos que en el proceso de mutación de D obtenemos que se debe rotar a partir del gen en la posición 5. Tendremos por lo tanto como resultado el siguiente D: (1,2,6,4,7,5,3,8)

6. A* y la búsqueda del camino mínimo

Existe un conjunto de problemas cuya resolución no sólo requieren encontrar un estado solución partiendo de un estado inicial. En ellos es importante conocer paso a paso los estados intermedios para llegar al destino. Generalmente, además, se busca el recorrido óptimo es decir el que minimice (o maximice según el problema) alguna magnitud obtenida durante el trayecto.

El espacio de estados se puede representar mediante un grafo y su recorrido exhaustivo en el peor de los casos resuelve el problema. El lector debe estar familiarizado, en este momento, con diversos algoritmos para enfrentar este tipo de problema. Algunos de ellos:

- Búsqueda por profundidad (Depth First Search): Con este algoritmo podemos encontrar un camino desde un estado origen hasta un estado destino
- Búsqueda por anchura (Breadth First Search): Con este algoritmo podemos encontrar un camino desde un estado origen a un estado destino utilizando la menor cantidad de estados intermedios.

-
- Algoritmo Dijkstra: Podemos encontrar un camino de costo mínimo en un grafo con ejes con costos positivos desde un origen a un destino.
 - Algoritmo Bellman-Ford: Podemos encontrar un camino de costo mínimo en un grafo con ejes con costos, y ausencia de ciclos negativos, desde un origen a un destino.

Todos estos algoritmos reciben el nombre de **algoritmos de búsqueda no-informada** (uninformed search algorithm). Por cada estado visitado se conoce el costo del recorrido desde el estado inicial, pero se desconoce qué tan cerca se encuentra del estado destino. Podemos pensar a estos como que buscan a ciegas el destino. En contraposición se encuentran los **algoritmos de búsqueda informada** que utilizan conocimiento relativo al dominio del problema para estimar la distancia aún no explorada al destino. El algoritmo que veremos a continuación forma parte de estos últimos.

En 1966 desde el Stanford Research Institute se comenzó a desarrollar a Shakey el primer autómatas de propósito general. Buscaban crear un robot capaz de razonar sobre sus propias acciones. Entre las funciones básicas se encontraba la posibilidad de físicamente trasladarse desde su ubicación a otra. Para lograrlo idearon el algoritmo que denominaron **A***¹⁰. Este algoritmo es muy similar a Dijkstra. Funciona también para grafos con costos positivos en los ejes.

Para analizar el funcionamiento del algoritmo A*, podemos dividir los nodos del grafo en 3 conjuntos: Los "alcanzados", los "frontera" y los "desconocidos". Al comenzar el único nodo alcanzado es el origen "i". Pertenecen a la "frontera" todos los nodos adyacentes a algún nodo "alcanzado" que no pertenezca a este último conjunto. Para cada nodo v de la frontera se calcula un valor de costo $f(v)$. Aquel con menor costo es seleccionado para pasar al conjunto de "alcanzados". Los adyacentes de este nodo en el conjunto de "desconocidos" pasan a formar parte del conjunto de la "frontera". Se actualizan los valores de costo de los nodos de la frontera. Este proceso se repite iterativamente hasta encontrar el nodo buscado o hasta que no queden nodos en el conjunto frontera para seleccionar.

¹⁰ P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," in IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, July 1968

En el algoritmo de Dijkstra la función $f(v)$ corresponde al costo del menor camino del nodo origen "i" a "v". Este valor corresponde al menor de los costos desde el origen de los nodos antecesores de "v" más el costo del eje que lo une a "v". Anteriormente hemos visto que Bajo las condiciones dadas, Dijkstra otorga el resultado óptimo al encontrar el camino al nodo destino. Más aún, la ejecución de Dijkstra hasta alcanzar todos los nodos nos asegura el camino mínimo a todos ellos.

La diferencia fundamental en A^* , es el cálculo de la función $f(v)$. Si llamamos $g(v)$ al cálculo realizado por Dijkstra, le sumaremos un nuevo término que llamaremos $h(v)$. Esta función corresponde a un valor heurístico que estima el costo del camino mínimo de "v" al nodo destino. Por lo tanto $f(v) = g(v) + h(v)$. El razonamiento detrás de la elección es que, es conveniente seleccionar ante diferentes opciones no sola la que me asegure el menor camino desde el origen, sino también aquella que nos acerca lo más posible al destino. Se espera que al seleccionar correctamente una heurística para el problema a enfrentar, ayude a acelerar el proceso de búsqueda, teniendo que visitar una menor cantidad de estados antes de finalizar. No esperamos encontrar el camino mínimo a todos los nodos, solo al nodo destino.

Veamos a continuación el pseudocódigo de A^* :

Camino mínimo - A^*
Sea i el nodo inicial sea d el nodo destino Sea $h(v)$ La función heurística que cada un nodo estima su distancia al nodo d frontera = $\emptyset$ costo[i]=0+h(i) y predecesor[i]=i Alcanzados = {(i,costo[i],predecesor[i])} costo[x]= ∞ para todo nodo x diferente a i Por cada nodo "s" accesible desde "i" costo[s] =C[i,s]+costo[i]-h(i)+h(s) predecesor[s]=i frontera = frontera + {"s", costo[s],predecesor[s]} Mientras la frontera $\neq \emptyset$ Sea "s" nodo en frontera con menor costo costo[s] Quitar s de frontera

Alcanzados = Alcanzados $\cup$ {s}

Si s es el nodo d

Retornar Costo[d] y camino de i a d.

Por cada x accesible desde "s" y $x \notin$ Alcanzados

nuevoCosto = costo[s] + C[s,x] - h(s) + h(x)

Si $x \in$ frontera y nuevoCosto < costo[x]

predecesor[x] = s

actualizar en frontera x con costo[x]=nuevoCosto

Si $x \notin$ frontera

predecesor[x] = s

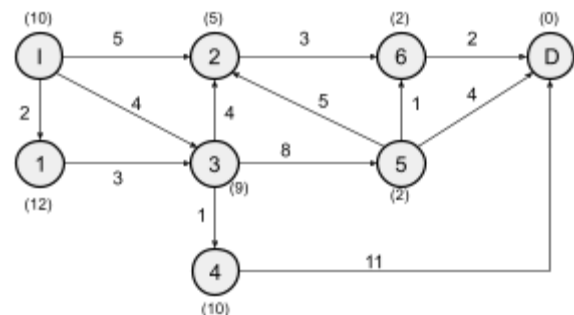
agregar en frontera x con costo[x]=nuevoCosto

Retornar 'El destino no es accesible'

Si consideramos la complejidad temporal de la función heurística $O(1)$ es fácilmente demostrable que la complejidad del algoritmo es igual a la de Dijkstra: $O((n + m) \cdot \log n)$ utilizando un Heap Binario. Donde "n" es la cantidad de nodos en el grafo y "m" el número de ejes.

En cuenta a la complejidad temporal también es igual a Dijkstra con $O(m+n)$. Obsérvese que no es necesario almacenar por cada nodo $g(v)$ y $h(v)$. Almacenando $f(v)$ y teniendo $h(v)$ se puede determinar el valor restante. Se debe considerar desde qué nodo se está accediendo y se lo puede calcular como $f(v) = c(u,v) + f[u] - h(u) + h(v)$. Donde $c(u,v)$ es el costo de utilizar el eje que conecta "u" con "v". Esta formulación es utilizada en el pseudocódigo para el cálculo del costo de cada nuevo nodo que se inserta en el conjunto frontera.

Consideremos el grafo de la imagen. Corresponde a un grafo dirigido con pesos positivos en los ejes. Los números entre paréntesis corresponden al valor heurístico de distancia de cada nodo al nodo destino "D". La búsqueda del camino mínimo comienza en el nodo "I". Calculamos su valor $f(I) = g(I) + h(I) = 0 + 10$. En la frontera tenemos a

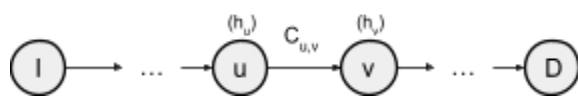


los nodos 1, 2 y 3. El camino de menor costo desde el origen a alguno de estos nodos es 2 al nodo 1. Sin embargo, considerando el valor heurístico y la fórmula $f(v) = c(u,v) + f[u] - h(u) + h(v)$ los

valores a evaluar al momento de la elección son 14, 10 y 13 respectivamente. El algoritmo A^* seleccionara expandir el nodo 2. Al hacerlo se une a la frontera el nodo 6 con un costo $f(6)=f(2)+c(2,6)-h(2)+h(6) = 10+3-5+2=10$. En la siguiente iteración podemos seleccionar alcanzar los nodos 1, 3 o 6. Este último tiene el menor costo posible y es elegido. Al hacerlo permite agregar a la frontera al nodo "D". Calculamos $f(D)=10+2-2+0=10$. En la última iteración seleccionamos el nodo D llegando al destino con costo 10. Podemos reconstruir el camino como $I \rightarrow 2 \rightarrow 6 \rightarrow D$. Corresponde efectivamente al menor camino posible para ese grafo.

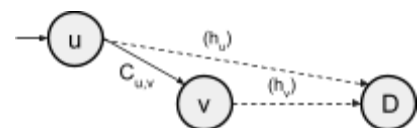
Definimos que un algoritmo de búsqueda es **completo** si encuentra un camino a la solución si este existe. Podemos ver que A^* lo es, dado que en el peor de los casos analizará todos los nodos hasta llegar al destino. Esto es factible si el número de nodos es finito. Sin embargo, la completitud no determina la optimalidad. El camino hallado podría no ser óptimo. ¿Qué condiciones debe verificar que lo sea? La única diferencia con Dijkstra y los requerimientos de optimalidad de este es el uso de la función heurística. Por lo tanto dependerá de esta.

Definimos que una heurística es **consistente** (consistent) si para todo nodo "u" del grafo, su costo estimado al destino $h(u)$ es menor, o igual, que la suma del costo estimado $h(v)$ y el costo de llegar de "u" a "v" $c(u,v)$ de cada uno de sus sucesores.



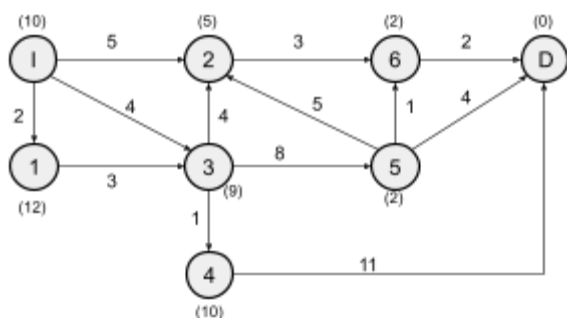
De forma compacta $h(u) \leq h(v) + c(u,v)$ para todo v sucesor de u. Esta propiedad también recibe el nombre de "monótona". Nos asegura que la primera vez que expandimos un nodo sea por haber llegado por el menor de sus caminos posibles. Para probarlo vemos que esta relación presenta una desigualdad triangular. Recordemos que a la hora de expandir evaluamos $f(u)=g(u)+h(u)$ y para cualquier sucesor "v" tenemos que $f(v)= c(u,v) + f[u] - h(u) + h(v)$. que podemos agrupar como $f(v) = f(u) + [h(v) + c(u,v)] - h(u)$.

Por ser consistente entonces la diferencia de los últimos dos términos será un valor positivo o cero. Por lo tanto vemos que $f(u) \geq f(v)$. Eso aplica para cualquier antecesor y para cualquier nodo. En conclusión a medida que



exploramos el grafo la función $f()$ sólo puede mantenerse o aumentar al transitar un camino al destino. Por lo tanto es una función monótona creciente. Dado como funciona el algoritmo, elegimos expandir entre los nodos disponibles a aquel con menor valor de $f()$. Al

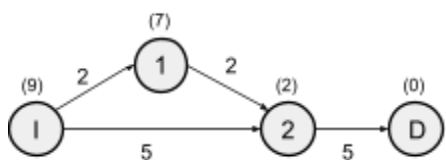
combinar estas dos situaciones, cuando expandimos un estamos seguros que hemos llegado al mismo por el menor camino posible. Por lo tanto probamos nuestra afirmación.



Observando el grafo del ejemplo anterior podemos verificar que la heurística $h()$ es consistente. Verificamos el cumplimiento de la desigualdad triangular $h(u) \leq h(v) + c(u,v)$ para el nodo "I", $h(I)=10$ y tiene como sucesores los nodos 1, 2 y 3. Para el primero tenemos $h(I) \leq h(1) + c(I,1)$. Reemplazando $10 \leq 12 + 2$. El segundo $h(I) \leq h(2) + c(I,2)$ con $10 \leq 5 + 5$. El último

tenemos $10 \leq 9 + 4$. Si repetimos la prueba con todos los nodos veremos que se cumple para cada uno de ellos esta desigualdad. Por lo que sabemos que el resultado obtenido será el óptimo al aplicar el algoritmo.

Si utilizamos una heurística inconsistente, usando el algoritmo descrito en pseudocódigo puede darnos un resultado no óptimo. Dado que la primera vez que expandimos un nodo puede no contener su camino mínimo hasta el. Para enfrentar este problema, se puede plantear modificar el algoritmo para permitir reabrir los nodos previamente alcanzados. Sin embargo, al hacerlo en el peor de los casos para obtener una solución requerimos una complejidad exponencial¹¹ y no tenemos aun así asegurado llegar a una solución óptima al llegar al destino.



El siguiente ejemplo no cumple con tener una heurística consistente. Lo verificamos con la desigualdad $h(I) \leq h(2) + c(I,2)$. El primer término tiene valor 9 y el segundo 7. Si ejecutamos A* al grafo iniciamos con el nodo I con $f(I)=9$.

Podemos seleccionar entre los nodos 1 y 2. Sus valores corresponden a $f(1)=9$ y $f(2)=7$. Selecciona expandir el nodo 2. Luego agregamos a la frontera el nodo "D" con $f(D) = c(2,D) + f(2) - h(2) + h(D) = 10$. En la frontera tenemos ahora los nodos 1 y D. Expandimos el nodo 1 por tener menor valor $f()$. Se agrega al conjunto de los alcanzados. Ningún sucesor de este nodo se encuentra en el conjunto de desconocidos. Por lo que se procede a seleccionar el

¹¹ Alberto Martelli. On the Complexity of Admissible Search Algorithms. Artificial Intelligence, 8(1):1-13, 1977

próximo nodo en la frontera con menor costo. Siendo el único el nodo D, se llega al destino encontrando el camino $I \rightarrow 2 \rightarrow D$ con costo total 5. Se puede comprobar fácilmente que no es el camino mínimo. Existe el camino $I \rightarrow 1 \rightarrow 2 \rightarrow D$ con costo 9.

Una propiedad para asegurarnos la optimalidad un poco más débil es la admisibilidad. Una heurística es **admisibile** (admissible) si nunca sobreestima el costo de llegar al destino. Es decir que para cualquier nodo u , $h(u) \leq h'(u)$. Donde $h'(u)$ es el menor costo real de llegar desde u al destino. Otra forma de nombrarla es "optimista". Además solicitamos que $h(D)=0$ con D el nodo destino.

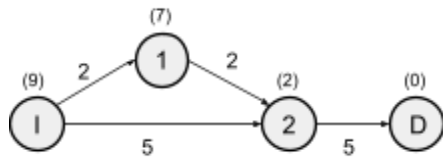
Se puede demostrar mediante inducción que si una heurística es consistente, entonces es admisible (pero no necesariamente a la inversa). Por lo tanto utilizar una heurística admisible (y a la vez inconsistente) podría, teniendo el pseudocódigo presentado, no brindar la solución óptima. Pero realizando la modificación de reabrir los nodos previamente alcanzados nos garantizará lograrlo. La modificación es pequeña: Al expandir un nodo, puede ocasionar la reducción de otro ya alcanzado en su costo de camino mínimo $g(n)$. De ocurrir lo quitaremos del conjunto de alcanzados y lo regresamos a la frontera con su nuevo valor $f(n)$ recalculado. Probemos la optimalidad de la solución encontrada. Lo realizaremos mediante una contradicción.

Supongamos que contamos con un grafo G y con una función heurística h admisible. Luego de la ejecución del algoritmo obtenemos como solución el camino C . Sin embargo existe un camino alternativo C' de menor costo total. Por nuestra hipótesis tenemos la siguiente relación de costos entre caminos: $|C| > |C'|$. Luego de la ejecución de algoritmo obtenemos $|C|$ como $f(D)=g(D)-h(D)=g(D)$ con D el nodo destino.

Por como funciona el algoritmo significa que al menos un nodo de C' se encuentra en la frontera y no fue seleccionado como alcanzado (o fue alcanzado pero por algún recálculo volvió a estar en la frontera). De lo contrario tendríamos que $|C|=|C'|$. Llamemos " u " a ese nodo. Además llamaremos $g'(u)$ y $h'(u)$ al costo del camino óptimo del inicio a " u " y al costo del camino óptimo de " u " al destino respectivamente (Estamos partiendo C' en dos partes). Como u está en la frontera al finalizar la ejecución, conocemos $f(u)=g(u)+h(n)$.

Podemos afirmar que $f(u) > C'$, de lo contrario como $|C'| < |C|$ el algoritmo hubiese expandido u . Por otro lado, sabemos que $g(u)=g'(u)$ dado que ya todos los nodos anteriores

de C^* a "u" se encuentran en el conjunto de alcanzados. Por lo tanto podemos mantener la igualdad reemplazando $f(u)=g'(u)+h(n)$. Como la heurística es admisible también podemos reemplazar por la inecuación $f(u)\leq g'(u)+h'(u)$. Como $g'(u)+h'(u)=C'$, entonces $f(u)\leq C'$. Esta última inferencia se contradice con $f(u)>C'$. Por lo tanto el razonamiento resulta absurdo y no es posible finalizar encontrando un camino de mayor costo al óptimo.



El gráfico del último ejemplo presenta una heurística admisible. El costo real de llegar al destino de cada nodo es igual o menor al valor heurístico. Consideremos el algoritmo modificado permitiendo reevaluar nodos previamente alcanzados. En un momento de la ejecución

expandimos el nodo 1 y lo pasamos al conjunto de alcanzados. En ese momento se estableció una nueva forma de acceder al nodo 2 previamente desconocida. Al momento de expandir el nodo por primera vez su valor era $f(2)=7$. Luego de expandir el nodo 1 podemos calcular $f(2)=c(1,2)+f(1)-h(1)+h(2)=2+9-7+2=6$ y lo volvemos a incluir en la frontera. El próximo paso encuentra dos nodos en la frontera 2 y D. El algoritmo seleccionará al nodo 2 por tener menor costo. Su posible sucesor es el D. Calculamos su costo $f(D)=9$. En la última iteración seleccionamos al nodo D y finalizamos encontrando el camino $I\rightarrow 1\rightarrow 2\rightarrow D$ con el costo 9 que resulta el óptimo.